30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 13

Data Layer Architecture — Part 1: Repositories, Models, Storage & Caching



Esmaeeil Moradi
@esmaeeil-moradi700

Why the data layer is Day 13

Day 3 introduced the repository as the single source of truth. Interviews go much further: *where does this data live, how fresh is it, who is allowed to write it, and what happens when it fails?*

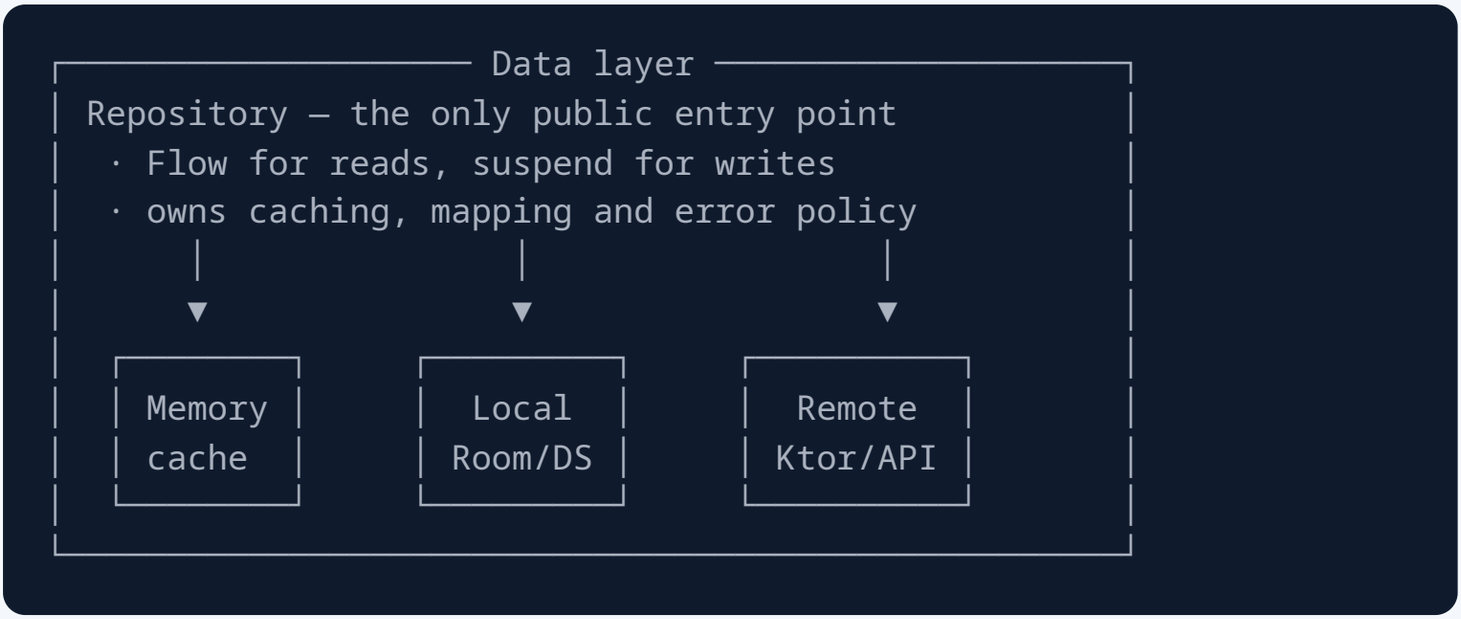
- Repository and data source design — the public API of your data
- Separate models per boundary, and mapping that protects the app
- Choosing storage: Room, DataStore, files — or nothing
- Caching strategies and **invalidation**
- Error handling and concurrency that survive real networks

Level	Questions
Junior	What a repository owns · Data sources · Memory vs disk cache
Mid	Models & mapping · Choosing storage · Caching strategies · Invalidation
Senior	Error handling · Concurrency in repositories · Consistency with many writers
Architect	Repository APIs at scale · Freshness policy · Defensive boundaries



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: one door into the data



- **Repositories** are the only door; data sources stay inside the data layer
- **Each data source** talks to exactly one source: a table, a file, an API
- **Policy lives in the repository:** freshness, mapping, errors, who may write

THE QUESTION BEHIND EVERY ANSWER TODAY

For any piece of data, you should be able to say: *who owns it, where is it stored, how stale may it be, and what does the user see when loading it fails?*



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 13

→

Junior

What a repository owns · Data sources · Memory vs disk cache



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What does a repository actually own?

Google's data layer guide gives repositories five jobs:

- **Expose data** to the rest of the app
- **Centralize changes** to the data
- **Resolve conflicts** between multiple data sources
- **Abstract** the sources of data away from the rest of the app
- **Contain business logic** about that data (e.g. "a draft is never uploaded")

Naming convention: *type of data* + `Repository` → `NewsRepository`, `PaymentsRepository`. One repository per **type of data**, not per screen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"A repository is a wrapper around Retrofit." — A pass-through wrapper adds nothing. The value is the **policy**: which source wins, how fresh data must be, and how errors and conflicts are resolved.

INTERVIEW TRAP

"One repository per screen." — Screens combine data from several repositories (in the ViewModel or a use case). A `ProfileScreenRepository` duplicates logic that `UserRepository` should own once.

FOLLOW-UP

"Can one repository depend on another?" — Yes, when data is layered (an `OrdersRepository` needing the current user). Keep the graph acyclic; when two repositories need each other, a use case above them is the fix.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Repositories and data sources

```
class NewsRemoteDataSource(
    private val api: NewsApi,
) {
    suspend fun fetchLatest(): List<NewsDto> = api.latest()
}

class NewsLocalDataSource(private val dao: NewsDao) {
    fun observeLatest(): Flow<List<NewsEntity>> = dao.observe()
    suspend fun replaceAll(items: List<NewsEntity>) =
        dao.replaceAll(items)
}

class NewsRepository(
    private val remote: NewsRemoteDataSource,
    private val local: NewsLocalDataSource,
) { /* combines both, owns the policy */ }
```

- A data source is **thin**: one source, no policy, no knowledge of other sources
- Name it *type of data* + *source*: `NewsRemoteDataSource`, `NewsLocalDataSource`



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Rules that keep the boundary clean

- ViewModels and use cases talk to **repositories only** — never to a DAO or an API service
- Data sources may expose storage models (`NewsEntity`, `NewsDto`); the repository maps them
- Interfaces where you need a seam (fakes in tests, several implementations) — not by reflex

INTERVIEW TRAP

"The ViewModel can call the DAO directly, it's faster." — Now two entry points write the same table, caching and mapping logic gets copied, and the single source of truth is gone.

INTERVIEW TRAP

"Every class needs an interface for testing." — A data source wrapping Room can often be tested with an in-memory database; an interface is worth it where the real thing is slow, flaky or remote.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. In-memory vs disk cache

	In-memory	Disk
Where	Heap: `StateFlow`, `Map`, `LruCache`	Room, DataStore, files, OkHttp cache
Survives	Configuration changes (if app-scoped)	Process death, reboot, app update
Speed	Fastest — no I/O	Needs I/O — keep it off the main thread
Size	Small — competes with the app's memory	Large — limited by storage
Use for	Hot data during a session	Anything you want to show offline or on cold start

Most apps use both: memory for the current session, disk as the durable copy, network to refresh.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Where memory caches go wrong

INTERVIEW TRAP

"I'll cache it in the ViewModel." — That cache is scoped to one screen and dies with it. An app-wide memory cache belongs in a **singleton-scoped repository**.

INTERVIEW TRAP

Writing your own bitmap `LruCache` (common v1-era advice). In 2026 image loaders like **Coil** or **Glide** already manage memory **and** disk caches with correct sizing — configure them instead.

FOLLOW-UP

"Is a memory cache thread-safe?" — Only if you make it so. Keep it in a `MutableStateFlow` updated with `update {}`, or guard a `Map` with a `Mutex` (Q9).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 13

Mid-Level

Models & mapping · Choosing storage · Caching strategies · Invalidation



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. One model per boundary

```
@Serializable                          // network contract
data class ArticleDto(
    val id: String,
    val title: String,
    @SerialName("published_at") val publishedAt: String,
)

@Entity(tableName = "articles")        // storage schema
data class ArticleEntity(
    @PrimaryKey val id: String,
    val title: String,
    val publishedAtEpochMs: Long,
)

data class Article(                    // what the app uses
    val id: ArticleId,
    val title: String,
    val publishedAt: Instant,
)
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Mapping — and when not to

```
fun ArticleDto.toEntity() = ArticleEntity(
    id = id,
    title = title,
    publishedAtEpochMs = Instant.parse(publishedAt)
        .toEpochMilliseconds(),
)

fun ArticleEntity.toDomain() = Article(
    id = ArticleId(id),
    title = title,
    publishedAt = Instant
        .fromEpochMilliseconds(publishedAtEpochMs),
)
```

- Each model changes for **its own reason**: the API, the schema, or the product
- Small app, no logic in between? Two models (DTO + entity) may be enough — be pragmatic

INTERVIEW TRAP

"Kotlin data classes are immutable by default" (v1 said this). — Only if every property is a `val` of an immutable type. `var` fields or a `MutableList` inside a data class are fully mutable.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Choosing local storage

Data	Choose	Why
Structured, queried, observed, relational	**Room**	SQL, indexes, transactions, `Flow` queries
Small settings, flags, tokens (Preferences)	**DataStore** Preferences	Async, transactional, `Flow`-based
One small typed object	**DataStore** with a serializer (Proto or JSON)	Type-safe, schema'd
Images, video, documents, large blobs	**Files** (`filesDir`, `cacheDir`)	Streaming; store the path in Room
Re-downloadable HTTP responses	**OkHttp cache**	Honors `Cache-Control` (Day 11)
Legacy key-value	`SharedPreferences`	Migrate it to DataStore

`cacheDir` may be cleared by the system when storage is low — never put data there that you can't re-create.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. The limits of each option

INTERVIEW TRAP

"DataStore is a small database." — DataStore rewrites its whole file on every update. Google's guidance: for large or complex data, **partial updates** or referential integrity, use Room.

INTERVIEW TRAP

Creating two DataStore instances for the **same file** in one process. It breaks DataStore's guarantees and throws when the file is read or written. Create it once (DI singleton or the `preferencesDataStore` delegate at top level).

INTERVIEW TRAP

Large BLOBs in Room rows. Rows too big for the cursor window fail to load and every query copies the bytes. Store the file on disk and the path in the table.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Caching strategies

Strategy	How it works	Good for
Stale-while-revalidate	Show cache now, refresh in background; the DB emits the update	Feeds, profiles, catalogs
Cache-first with TTL	Use the cache while it's younger than *X*; otherwise fetch	Config, reference data
Network-first, cache fallback	Try the network; on failure show the cache, marked stale	Prices, availability
Network-only	Never persist	Search-as-you-type, OTPs, payment status
Cache-only, filled by sync	UI reads only the DB; background sync writes it	Offline-first apps (Day 14)

Choose **per data type**, not per app.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Stale-while-revalidate with Room as SSOT

```
class NewsRepository(
    private val local: NewsLocalDataSource,
    private val remote: NewsRemoteDataSource,
    private val clock: Clock,
) {
    fun observeNews(): Flow<List<News>> = local.observeNews()

    suspend fun refreshIfStale(maxAge: Duration = 15.minutes) {
        val last = local.lastFetchedAt()
        if (last != null && clock.now() - last < maxAge) return
        val fresh = remote.fetchNews()
        local.replaceAll(fresh, fetchedAt = clock.now())
    }
}
```

The UI collects `observeNews()`; the ViewModel calls `refreshIfStale()` separately. Cached rows appear instantly, fresh rows arrive through the **same** `Flow`. An injected `Clock` makes the TTL testable.

INTERVIEW TRAP

v1's flow: check memory → check DB → call network → *"return the fresh data to the ViewModel"*. A one-shot result that bypasses the DB creates two paths to the UI — and misses every later change.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Cache invalidation

Trigger	Example
Time (TTL)	Catalog older than 1 hour → refresh on next open
User action	Pull-to-refresh forces a fetch
Server signal	Data-only push message: "orders changed" → schedule a refresh
Local write	After posting a comment, refresh (or patch) the thread
Identity change	Logout or account switch → **wipe** user-scoped data
Schema/contract change	App update changes the model → migrate or drop the cache
Size	LRU eviction for memory and image caches

Invalidating means marking data stale or deleting it — the UI still reads from the same source.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Logout must wipe everything

```
class UserDataCleaner(
    private val db: AppDatabase,
    private val prefs: DataStore<Preferences>,
    private val httpCache: Cache,
    private val io: CoroutineDispatcher,
) {
    suspend fun wipe() = withContext(io) {
        db.clearAllTables()      // blocking: never on main
        prefs.edit { it.clear() }
        httpCache.evictAll()     // OkHttp disk cache
        // + user files, image caches, scheduled work
    }
}
```

INTERVIEW TRAP

"Logout = delete the token." — The next account on that phone then sees the previous user's orders, cached images and HTTP responses. Every user-scoped store must be cleared — or keyed by user ID.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 13

Senior

Error handling · Concurrency in repositories · Consistency with many writers



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Error handling across the data layer

```
sealed interface DataError {
    data object Offline : DataError
    data class Http(val code: Int) : DataError
    data object BadData : DataError
}

suspend fun <T> safeCall(block: suspend () -> T): Outcome<T> =
    try {
        Outcome.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Outcome.Err(DataError.Offline)
    } catch (e: HttpException) {
        Outcome.Err(DataError.Http(e.code()))
    } catch (e: SerializationException) {
        Outcome.Err(DataError.BadData)
    }
```

Data sources throw; the repository **translates** low-level exceptions into errors the app understands. Callers never import Retrofit, Ktor or SQLite types.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Rules for errors

- Classify: **transient** (offline, timeout, 5xx → retry later), **permanent** (4xx validation → tell the user), **auth** (401 → token refresh, Day 12)
- Refresh failed but cache exists? Keep showing data **plus** a message — not a full-screen error
- A `catch` operator on a `Flow` only catches **upstream** errors, and the flow ends after it

INTERVIEW TRAP

"Kotlin's `Result` can't be a return type." — Outdated: that restriction was lifted in Kotlin 1.5. The real question is whether `Throwable` is precise enough — a sealed error type is usually clearer.

INTERVIEW TRAP

"Retry every failed request." — Retrying a non-idempotent `POST` can create two orders. Retry only idempotent calls, or calls carrying an idempotency key (Day 14).



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Concurrency inside a repository

```
class ProfileRepository(
    private val remote: ProfileRemoteDataSource,
    private val externalScope: CoroutineScope, // app-scoped
) {
    private val mutex = Mutex()
    private var cached: Profile? = null

    suspend fun profile(refresh: Boolean = false): Profile {
        if (!refresh) {
            mutex.withLock { cached }?.let { return it }
        }
        return externalScope.async {
            val fresh = remote.fetch()
            mutex.withLock { cached = fresh }
            fresh
        }.await()
    }
}
```

- `Mutex` guards the cache; the lock is **not** held during the network call
- `externalScope.async { }.await()`: if the screen leaves, the caller is cancelled but the fetch finishes and **fills the cache** (pattern from Google's data layer guide)



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What interviewers probe

- **Main-safety:** the repository never blocks the caller. Room and Retrofit/Ktor `suspend` APIs are already main-safe
- **Work that must finish** (saving a draft, a payment request): run it in an app-scoped `CoroutineScope` injected by DI — or WorkManager if it must survive process death
- **Duplicate requests:** ten callers → ten fetches. Share one in-flight `Deferred`, or refresh through one place

INTERVIEW TRAP

`GlobalScope.launch` for "fire and forget" writes. No owner, no cancellation, untestable. Inject an application `CoroutineScope` so tests can control it.

INTERVIEW TRAP

`withContext(NonCancellable)` around the whole operation. It hides cancellation everywhere; use it only for short cleanup that must run after cancellation.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Consistency when many writers touch one table

```
@Dao
abstract class ArticleDao {
    @Query("DELETE FROM articles WHERE feed = :feed")
    abstract suspend fun deleteFeed(feed: String)

    @Upsert
    abstract suspend fun upsertAll(items: List<ArticleEntity>)

    @Transaction
    open suspend fun replaceFeed(
        feed: String,
        items: List<ArticleEntity>,
    ) {
        deleteFeed(feed)
        upsertAll(items)
    }
}
```

One transaction → observers never see the empty table in between, and a crash leaves the old data intact.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Races between network, user and sync

- **Stale response overwrites a newer edit:** guard writes with a version, e.g. `WHERE version < :newVersion`
- **Refresh vs local edit:** don't overwrite rows that still have pending local changes (Day 14: outbox)
- Room re-runs an observed query when **its table** changes — add `distinctUntilChanged()` to skip identical emissions

INTERVIEW TRAP

"Wrap the Room update and the network call in one transaction" (v1). — Impossible: a DB transaction can't include a remote server. Holding a write transaction open during network I/O also blocks every other writer. Commit locally, then sync.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 13

Architect

Repository APIs at scale · Freshness policy · Defensive boundaries



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Designing repository APIs at scale

```kotlin
// :core:data — the public contract
interface NewsRepository {
    fun observeNews(topic: TopicId): Flow<List<News>>
    suspend fun refresh(topic: TopicId): Outcome<Unit>
    suspend fun setBookmarked(id: NewsId, saved: Boolean)
}

// Implementation and data sources stay module-private
internal class OfflineFirstNewsRepository(
    private val dao: NewsDao,
    private val api: NewsApi,
) : NewsRepository { /* … */ }
```

Module layout in the style of Now in Android: `:core:network`, `:core:database`, `:core:datastore` → `:core:data` → features. Features see the interface, never a DAO.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The decisions behind the API

Decision	Prefer	Avoid
Granularity	One repository per data type	One per screen; a god repository
Reads	`Flow` of domain models	`suspend get()` for data that changes
Writes	`suspend` returning an outcome	Fire-and-forget from UI scope
Ownership	One repository writes each table	Two repositories writing the same rows
Shape	Domain verbs: `setBookmarked()`	Generic `BaseRepository<T>` CRUD

INTERVIEW TRAP

A generic `BaseRepository<T>` with `getAll/insert/delete`. It leaks storage semantics into the domain and fights every rule that isn't CRUD (drafts, soft deletes, permissions).



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Freshness is a product decision

Data	Policy	User sees
Feature flags, config	Cache-first, long TTL, refresh at start	Last known config
Feed, catalog	Stale-while-revalidate	Instant content, quiet update
Profile, settings	Local-first, synced writes	Own edits immediately
Balance, stock, price	Network-first, cache as fallback	Value **with its age** ("as of 10:42")
Search results	Memory only	Nothing persisted
Secrets, OTPs	Never cached	—

- Write the policy down **per data type** (an ADR or a table in the repo), then implement it in the repository
- Showing old data is fine when you **say** it's old; silently stale money or stock is a bug

INTERVIEW TRAP

"Make everything offline-first." — Some data must not be shown stale without a label, and some must never be persisted. Offline-first is a choice per data type (Day 14).



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Defensive boundaries: the contract will change

```kotlin
@Serializable
data class OrderDto(
    val id: String? = null,
    val status: String? = null,
    val totalCents: Long? = null,
)

fun OrderDto.toDomainOrNull(): Order? {
    val id = id ?: return null
    val total = totalCents ?: return null
    val status = when (status) {
        "pending" -> OrderStatus.Pending
        "paid" -> OrderStatus.Paid
        else -> OrderStatus.Unknown     // new server value
    }
    return Order(OrderId(id), status, Money(total))
}
```

`dtos.mapNotNull { it.toDomainOrNull() }` — one bad item is dropped (and logged), not the whole screen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Old app versions live for years

- Old versions stay installed long after you ship: every API change must be **backward compatible** for them
- The mapper is an **anti-corruption layer**: the domain never sees nullable junk or unknown enums
- Persisted data is a contract too: Room rows, DataStore files, WorkManager input and queued payloads written by v1 are read by v2 (Day 14: migrations)

INTERVIEW TRAP

A strict enum in the DTO. By default kotlinx.serialization throws on an unknown enum value — one new server status crashes parsing of the whole list for every old install. Map from `String`, or add a fallback.

SENIOR ANSWER IN ONE LINE

"Repositories are the only door to data; each type of data has one owner, an explicit freshness policy, its own models, and errors translated at the boundary."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 13 · ARCHITECT

Day 13 — cheat sheet

- A repository exposes data, centralizes changes, resolves source conflicts and owns the policy. **One per data type**, not per screen.
- Data sources are thin: one source each, no policy, invisible outside the data layer.
- Memory cache for the session (app-scoped), disk for offline and cold start; images → Coil/Glide.
- Separate DTO / entity / domain models where they change for different reasons; `data class` ≠ immutable.
- Room for structured data, DataStore for small settings (no partial updates), files for blobs.
- Pick a caching strategy per data type; the UI always reads from **one** source.
- Invalidate by TTL, user action, server signal, write — and **wipe everything** on logout.
- Translate exceptions at the repository; rethrow `CancellationException`; only retry idempotent calls.
- App-scoped work goes in an injected `externalScope`; guard caches with `Mutex`; no network inside DB transactions.
- Map defensively: unknown values and missing fields must not crash old app versions.

Day 14 is next → Data Layer Part 2: offline-first, sync & scale. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700